Universidad Tecnológica Nacional

Tecnicatura Universitaria en Programación a Distancia

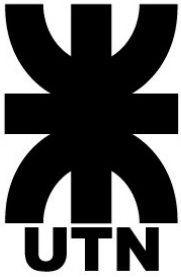
Metodología de Sistemas II

Trabajo Práctico Integrador (TPI)

Evolución profesional de un sistema existente

Sistema de Gestión de Solicitudes Académicas

Integrantes	Santiago Nievas / Ignacio Figueroa
Año	2026
Materia	Metodología de Sistemas II
Modalidad	Grupal
Comisión	[COMPLETAR_COMISION]
Fecha	4 de noviembre de 2026



1. Resumen ejecutivo

El trabajo analiza y mejora un Sistema de Gestión de Solicitudes Académicas desarrollado en Python. La versión inicial cumple las operaciones visibles requeridas, pero concentra persistencia, reglas de negocio, búsqueda y reportes en una única clase, generando deuda técnica. Se identificaron diez debilidades de diseño y se priorizaron las que afectan mantenibilidad, testabilidad y costo del cambio. La versión refactorizada separa dominio, servicios e infraestructura, introduce objetos de dominio, contratos de repositorio y manejo explícito de errores. Se aplicaron Extract Method, reubicación de responsabilidad, Introduce Parameter Object, Extract Class y reemplazo de strings mágicos por enums. Una suite automatizada de seis pruebas, incluida una comparación antes/después, verifica que el comportamiento externo nominal se conserva. Finalmente, se realizó un code review con criterios técnicos y se dejó trazabilidad Git por etapas para respaldar las decisiones del equipo.

Índice de contenido

1. Resumen ejecutivo

2. Descripción del sistema base y alcance

3. Calidad externa vs. calidad interna

4. Diagnóstico de deuda técnica y code smells

5. Aplicación de SOLID y Clean Code

6. Refactorings aplicados y evidencia antes/después

7. Pruebas y verificación del comportamiento

8. Code review profesional

9. Repositorio, trazabilidad y participación

10. Uso de inteligencia artificial

11. Conclusiones y mejoras futuras

12. Anexos y enlaces

2. Descripción del sistema base y alcance

Dominio. El caso seleccionado es un Sistema de Gestión de Solicitudes Académicas. Su finalidad es registrar pedidos de estudiantes, clasificarlos, asignar responsables, actualizar su estado, consultarlos y producir un resumen operativo. Se eligió un dominio acotado para concentrar la evaluación en calidad y evolución del código, no en cantidad de funcionalidades.

Actores. El estudiante origina una solicitud; el personal administrativo registra o consulta información; y un responsable de área toma la solicitud y actualiza su estado hasta su resolución o rechazo.

Tecnología. Se utilizó Python 3.11+ y persistencia JSON. La versión final no depende de librerías externas. Las pruebas se implementaron con unittest, lo que permite ejecutar la entrega en un entorno estándar.

Elemento	Definición
Entidades / estructuras principales	Solicitud, Estudiante, Responsable/Área y Estado.
Operaciones verificables	Registrar; asignar responsable; cambiar estado; buscar/filtrar; generar reporte.
Persistencia	Archivo JSON mediante un repositorio de infraestructura.
Versión inicial	`version_inicial/sistema.py`, tag `estado-inicial`.
Versión final	`version_refactorizada/`, tag `estadorefactorizado`.
Repositorio GitHub	[ENLACE_REPOSITORIO]

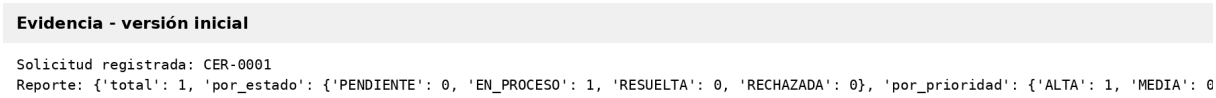


Figura 1. Ejecución de la versión inicial: el flujo principal funciona desde la mirada del usuario.

Alcance. El proyecto no implementa autenticación, interfaz web ni integración con sistemas institucionales reales. Los datos usados son sintéticos. Esas exclusiones evitan desviar el trabajo hacia funcionalidad que no aporta evidencia adicional a los objetivos de la materia.

3. Calidad externa vs. calidad interna

Desde la calidad externa, el sistema base permite completar el recorrido esperado: registrar una solicitud, asignarla, cambiar su estado, filtrarla y obtener un reporte. No se parte de un sistema “roto”, sino de un sistema funcional cuya estructura interna dificulta el mantenimiento.

Perspectiva	Situación observada	Consecuencia
Calidad externa	Las operaciones principales producen resultados correctos en el flujo nominal.	El usuario puede utilizar el módulo y percibe funcionalidad suficiente.
Calidad interna	Una clase concentra reglas, persistencia y reportes; abundan strings, condicionales y datos primitivos.	El equipo técnico paga un mayor costo para entender, probar y modificar.
Mantenibilidad	Los cambios atraviesan una clase central y obligan a conocer detalles de JSON y reglas de negocio al mismo tiempo.	Aumenta el riesgo de regresiones y el tiempo de evolución.
Testabilidad	La dependencia directa del archivo obliga a preparar persistencia incluso para probar reglas simples.	Las pruebas resultan más frágiles y lentas de escribir.

La comparación permite mostrar el punto central del TPI: dos versiones pueden ofrecer una calidad externa similar y, sin embargo, tener costos de cambio muy distintos. La mejora se concentra en la estructura interna y en la capacidad de verificarla con pruebas.

4. Diagnóstico de deuda técnica y code smells

La deuda técnica aparece porque la versión inicial prioriza resolver el flujo rápidamente a costa de una estructura centralizada. El problema no es que cada smell sea un error funcional inmediato, sino que actúan como señales de diseño que incrementan el esfuerzo futuro.

#	Problema	Evidencia / ubicación	Impacto o riesgo	Prioridad
1	Large Class	`SistemaSolicitudes` carga, guarda, valida, registra, busca y reporta.	Baja cohesión; cambio con efecto dominó.	Alta
2	Long Method	`registrar()` valida, normaliza, genera ID/código, calcula puntaje, crea y persiste.	Lectura y test aislado difíciles.	Alta
3	Long Parameter List	`registrar()` recibe 8 parámetros.	Orden frágil y evolución incómoda.	Alta
4	Primitive Obsession	Estudiante, responsable, estado, tipo y prioridad son strings/dicts.	Valores inválidos y reglas dispersas.	Alta
5	Duplicated Code	Búsqueda/actualización y persistencia se repiten en operaciones.	Inconsistencias al cambiar reglas.	Media
6	Magic Strings	`PENDIENTE`, `ALTA`, `CERTIFICADO`, áreas, etc.	Errores tipográficos y validación repetida.	Alta
7	Conditional Complexity	Cadenas `if/elif` para prefijos, prioridad y filtros.	Agregar variantes obliga a editar código existente.	Media
8	Error Silencing	`except Exception` reemplaza cualquier error por una lista vacía.	Oculta corrupción o fallos de I/O.	Alta
9	Tight Coupling	La lógica depende directamente de `Path`, JSON y archivo físico.	Dificulta pruebas y cambio de persistencia.	Alta
10	Responsabilidad mal ubicada	El reporte agregado vive en la misma clase que registra solicitudes.	La clase crece por motivos diferentes.	Media

Código 1. Fragmento representativo de la versión inicial: lista extensa de parámetros, condicionales y múltiples responsabilidades.

```
def registrar(self, legajo, nombre, email, tipo, prioridad, descripcion, area, responsable=""):
  if not legajo or not nombre or not email or "@" not in email or not descripcion:
    return None
  ...
  if tipo == "CERTIFICADO":
    codigo = "CER-" + str(numero).zfill(4)
  elif tipo == "EQUIVALENCIA":
    codigo = "EQU-" + str(numero).zfill(4)
  ...
  self.s.append(x)
  self.guardar()
```

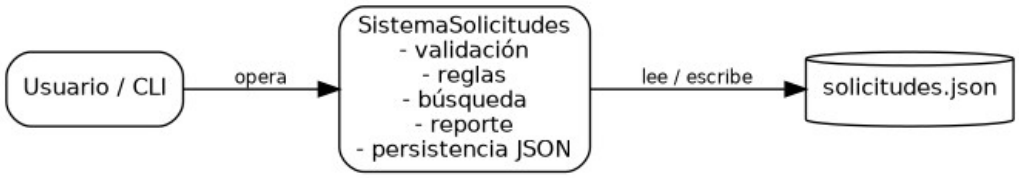


Figura 2. Arquitectura inicial: una única clase concentra el comportamiento y la persistencia.

Priorización. Se consideran críticos los problemas que afectan simultáneamente a mantenibilidad y testabilidad: clase grande, método largo, parámetros extensos, primitive obsession, manejo silencioso de errores y acoplamiento a persistencia. Los condicionales y la duplicación se corrigen en el mismo proceso porque agravan el costo de extensión.

5. Aplicación de SOLID y Clean Code

Las mejoras no se aplicaron como una lista mecánica de principios. Cada principio se vinculó con una debilidad concreta de la versión inicial y se verificó en la estructura resultante.

Principio	Aplicación en la versión final	Efecto
SRP	`SolicitudService` coordina casos de uso; `ReporteService` reporta; el repositorio persiste; las entidades conservan reglas propias.	Reduce razones de cambio por clase.
OCP	Los servicios dependen de contratos de repositorio y admiten nuevas implementaciones sin modificarse. El soporte de nuevos tipos de solicitud todavía requiere ampliar el enum, por lo que se considera una mejora parcial, no una aplicación absoluta.	Evita sobreafirmar el principio y concentra extensiones donde corresponden.
LSP	`JsonSolicitudRepository` e `InMemorySolicitudRepository` pueden sustituirse bajo el mismo contrato en los servicios.	Permite cambiar infraestructura sin alterar los casos de uso.
ISP	Se separan `SolicitudReader` y `SolicitudWriter`; `ReporteService` necesita solo lectura.	Un consumidor no depende de operaciones que no usa.
DIP	`SolicitudService` y `ReporteService` dependen de Protocols, no de la clase JSON concreta.	Disminuye acoplamiento y facilita pruebas.

Práctica Clean Code	Aplicación	Resultado
Nombres del dominio	`Solicitud`, `Estudiante`, `Responsable`, `EstadoSolicitud`, `NuevaSolicitudData`.	El código comunica intención sin depender de comentarios explicativos.
Funciones pequeñas	El flujo de alta delega en `_siguiente_id()` y `_crear_solicitud()`.	Cada método trabaja en un nivel de abstracción más acotado.
Errores explícitos	`DatosInvalidosError` y `SolicitudNoEncontradaError`.	Se evita retornar valores ambiguos o silenciar excepciones.

Alta cohesión	Las reglas de una solicitud viven en `Solicitud`; el reporte en su servicio.	Cada componente reúne responsabilidades relacionadas.
Bajo acoplamiento	La aplicación conoce contratos de repositorio.	La infraestructura puede cambiar con menor impacto.

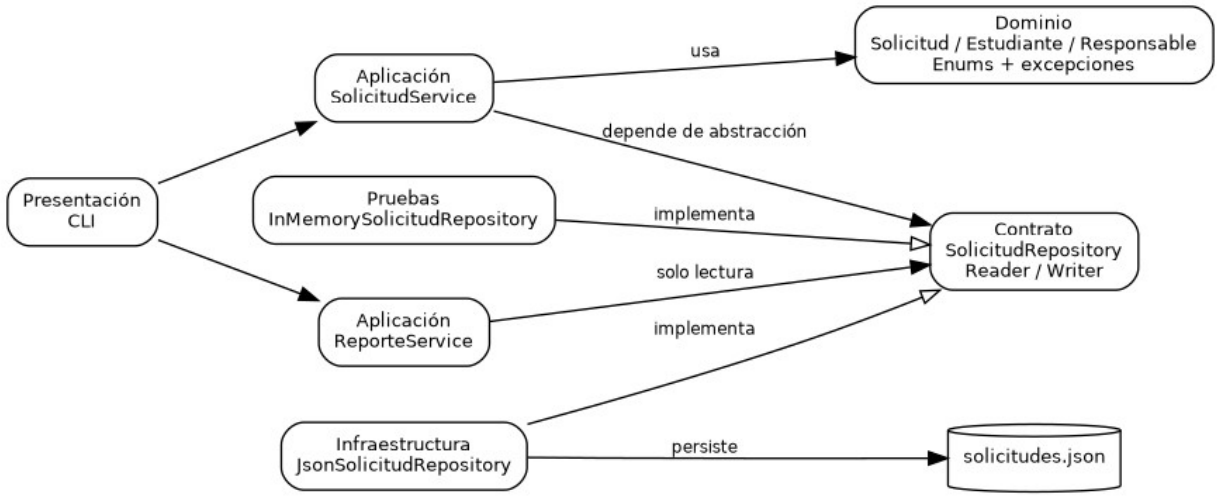


Figura 3. Arquitectura final: dominio, aplicación e infraestructura separados por responsabilidades y contratos.

6. Refactorings aplicados y evidencia antes/después

Refactoring	Antes	Después	Beneficio
Extract Method	`registrar()` concentra tareas.	Se extraen `_siguiente_id`, `_crear_solicitud` y `_obtener_para_actualizar`.	Flujo principal más corto y testeable.
Move Method / reubicación	El reporte está dentro de `SistemaSolicitudes`.	Se mueve a `ReporteService`, que solo necesita `SolicitudReader`.	Mejora SRP y cohesión.
Introduce Parameter Object	El alta recibe 8 parámetros.	`NuevaSolicitudData` agrupa los datos de entrada.	Firma estable y legible.
Extract Class + DIP	Persistencia JSON mezclada con negocio.	Se extrae `JsonSolicitudRepository` y se introduce contrato.	La aplicación deja de depender del archivo.
Replace Magic Strings	Estados, tipos y prioridades son strings.	Se crean `TipoSolicitud`, `Prioridad` y `EstadoSolicitud`.	Menos valores inválidos y lógica duplicada.

Código 2. Después del Extract Method e Introduce Parameter Object: el caso de uso principal queda expresado en pocos pasos.

```
def registrar(self, data: NuevaSolicitudData) -> Solicitud:
    solicitudes = self.repository.list_all()
    solicitud_id = self._siguiente_id(solicitudes)
    solicitud = self._crear_solicitud(solicitud_id, data)
    solicitudes.append(solicitud)
    self.repository.save_all(solicitudes)
    return solicitud
```

Código 3. Parameter Object que reemplaza la lista extensa de parámetros.

```
@dataclass(frozen=True)
class NuevaSolicitudData:
    legajo: str
    nombre: str
    email: str
    tipo: TipoSolicitud
    prioridad: Prioridad
    descripcion: str
    area: str = "SECRETARIA_ACADEMICA"
```

Refactoring seguro. La regla seguida fue cambiar la estructura interna sin alterar negativamente el comportamiento externo. Para sostener esa afirmación, no se utilizó solamente inspección visual: se automatizó una prueba de equivalencia que ejecuta el mismo caso de uso en ambas implementaciones y compara los datos funcionales y el reporte resultante.

7. Pruebas y verificación del comportamiento

La estrategia de verificación combina pruebas de la versión inicial, pruebas específicas de la versión final y una prueba de no regresión entre ambas. Esto permite distinguir dos preguntas: “¿la nueva arquitectura funciona?” y “¿sigue haciendo lo que hacía el sistema base?”.

ID	Caso	Resultado esperado	Estado
T01	Registrar solicitud válida	Código `CER-0001`, normalización de email y estado inicial correcto.	OK
T02	Persistir y recargar versión inicial	La solicitud guardada se recupera sin pérdida funcional.	OK
T03	Flujo final: alta + asignación + estado + filtros + reporte	Resultados consistentes con el dominio.	OK
T04	Email inválido	Se lanza `DatosInvalidosError`.	OK
T05	Solicitud inexistente	Se lanza `SolicitudNoEncontradaError` .	OK
T06	Equivalencia antes/después	Mismas claves funcionales y mismo reporte para el flujo nominal.	OK

Evidencia - ejecución de pruebas

```
test_mismo_resultado_externo_en_flujo_nominal (test_equivalencia.TestEquivalenciaAntesDespues.test_mismo_resultado_externo_en_flujo_nominal)
test_flujo_principal (test_version_inicial.TestVersionInicial.test_flujo_principal) ... ok
test_persistencia (test_version_inicial.TestVersionInicial.test_persistencia) ... ok
test_error_explicito_si_no_existe (test_version_refactorizada.TestVersionRefactorizada.test_error_explicito_si_no_existe) ... ok
test_flujo_principal (test_version_refactorizada.TestVersionRefactorizada.test_flujo_principal) ... ok
test_validacion_de_email (test_version_refactorizada.TestVersionRefactorizada.test_validacion_de_email) ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.004s

OK
```

Figura 4. Suite automatizada: seis pruebas ejecutadas sin fallos.

Comando reproducible desde la raíz del repositorio.

```
python -m unittest discover -s tests -v
```

La prueba T06 es la principal evidencia de seguridad del refactoring: compara la salida funcional de una solicitud de equivalencia procesada en la versión inicial y en la final. El objetivo no es demostrar equivalencia matemática para cualquier entrada, sino documentar una verificación repetible sobre los comportamientos representativos del módulo.

8. Code review profesional

El code review se trató como una revisión colaborativa y técnica, no como una evaluación personal. El checklist cubre lógica, legibilidad, diseño, seguridad, rendimiento proporcional al caso y pruebas. Las observaciones se convierten en acciones y criterios de aceptación verificables.

Criterio	Resultado	Observación
Lógica y funcionalidad	Cumple	Alta, asignación, estado, filtros y reporte cubiertos por pruebas.
Casos de borde	Cumple	Email inválido y solicitud inexistente tienen comportamiento explícito.
Nombres descriptivos	Cumple	Se utilizan términos del dominio académico.
Funciones pequeñas	Cumple	Se extrajeron responsabilidades del método de alta.
SRP / cohesión	Cumple	Dominio, aplicación, persistencia y reporte separados.
DIP / acoplamiento	Cumple	Servicios dependen de contratos.
ISP	Cumple	Lectura y escritura separadas para consumidores específicos.
LSP	Cumple	Repositorios JSON e InMemory son sustituibles en el servicio.
OCP	Parcial	La infraestructura es extensible; nuevos tipos aún amplían el enum.
Manejo de errores	Cumple	No se silencian excepciones en la versión final.
Seguridad / privacidad	Cumple	Datos sintéticos y sin secretos.
Tests	Cumple	6 pruebas automatizadas, incluida no regresión.

Feedback aplicado. Las observaciones principales fueron reducir la clase central, eliminar strings libres para conceptos de dominio, reemplazar la firma extensa del alta, desacoplar la persistencia y hacer visible el manejo de errores. Todas ellas tienen una corrección identificable en la versión final.

Criterios de aceptación técnica. Para aprobar la revisión: la suite debe pasar; el flujo nominal debe conservar comportamiento; no deben existir credenciales ni datos personales reales; las dependencias deben respetar los límites definidos; y los tags de trazabilidad deben estar presentes.

9. Repositorio, trazabilidad y participación

El repositorio local fue construido con commits por etapa y los tags solicitados por la consigna. La secuencia permite recuperar el estado base, el diagnóstico, el refactoring, las pruebas y la documentación, en lugar de presentar únicamente una fotografía final.

Hito	Tag	Contenido principal
Estado base	estado-inicial	Versión inicial funcional.
Diagnóstico	etapa-diagnostico	Code smells, impacto y prioridades.
Refactoring	etapa-refactoring	Dominio, servicios, repositorios y objetos de entrada.
Pruebas	etapa-pruebas	Pruebas iniciales, finales y equivalencia.
Documentación	etapa-docs	Code review, IA, defensa y evidencias.
Estado final	estadorefactorizado	Versión final preparada para publicación.

Evidencia - historial Git por etapas

```
* 54c0287 (HEAD -> main, tag: etapa-docs, tag: estadorefactorizado) docs: completar code review trazabilidad y guia de defensa
* 1d9f0fa (tag: etapa-pruebas) test: verificar comportamiento y equivalencia antes despues
* dfe914d (tag: etapa-refactoring) refactor: separar dominio servicios y persistencia
* 2f3f6b3 (tag: etapa-diagnostico) docs: registrar diagnostico de calidad interna y code smells
* 8202715 (tag: estado-inicial) feat: incorporar version inicial funcional del sistema de solicitudes
```

Figura 5. Historial Git local organizado por etapas del TPI.

Participación. El trabajo se presenta de forma grupal por Santiago Nievas e Ignacio Figueroa. Para la defensa se distribuye la exposición: Santiago presenta sistema base, calidad y deuda técnica; Ignacio explica SOLID, refactorings, pruebas y code review; ambos participan en la demostración y conclusiones. Antes de publicar, la trazabilidad del repositorio debe reflejar participación real de ambos integrantes y no solamente una división declarativa.

10. Uso de inteligencia artificial

Herramienta utilizada: ChatGPT (OpenAI). Se utilizó como apoyo para estructurar el caso, explorar alternativas de refactoring, preparar documentación técnica, diseñar casos de prueba y revisar consistencia con la consigna y los materiales de la asignatura.

Grado de intervención. La herramienta produjo propuestas y borradores que se materializaron en código y documentación verificables. La validación no se basó en aceptar texto de forma automática: la suite se ejecutó, se comparó el comportamiento antes/después y se revisó la estructura contra los criterios técnicos de la materia.

Registro del prompt principal utilizado en la interacción de IA.

> Quiero que resuelvas el TPI de Metodología, te voy a pasar las consignas y contexto para que puedas resolverlo y entregarme un zip con todo lo necesario. Además quiero hacerlo con mi compañero Ignacio Figueroa.

Trazabilidad. El anexo incluye `USO_DE_IA.md` y este informe registra la finalidad, el alcance y la validación. La defensa oral deberá realizarse con voz propia y ambos integrantes deben comprender y poder justificar lo entregado.

11. Conclusiones y mejoras futuras

Conclusión grupal. El caso permitió comprobar que un sistema puede cumplir su función y, aun así, acumular una estructura costosa de modificar. La mejora más relevante no fue agregar funcionalidades, sino reducir razones de cambio, expresar conceptos del dominio y crear límites claros entre reglas e infraestructura. Las pruebas dieron seguridad para modificar el diseño y el code review transformó observaciones subjetivas en criterios verificables.

Dificultades y límites. El principal desafío fue mejorar la estructura sin convertir un módulo simple en una arquitectura innecesariamente compleja. Por eso se mantuvo una solución pequeña, con contratos solo donde aportan testabilidad o desacoplamiento. También se reconoce que OCP no es "perfecto": incorporar un tipo de solicitud exige ampliar el enum; la mejora se concentra en que la infraestructura y los servicios no deban reescribirse por cambios de persistencia.

Mejoras futuras. Como evolución se podría incorporar una API web, autenticación por roles, base de datos relacional, auditoría de cambios y filtros compuestos. La arquitectura final permite realizar esas extensiones sin devolver la lógica de negocio a una clase monolítica.

Conclusión individual - Santiago Nievas. El aprendizaje principal es que la mantenibilidad se puede observar en decisiones concretas: tamaño de métodos, responsabilidades, dependencias y capacidad de probar. El refactoring resultó más defendible cuando cada cambio quedó asociado a un problema previo y a una prueba de verificación.

Conclusión individual - Ignacio Figueroa. El aporte central del proceso es la trazabilidad: diagnóstico, cambio, prueba y revisión forman una misma cadena. Un refactoring aislado puede verse como una preferencia de estilo; con evidencia antes/después y criterios de aceptación se convierte en una mejora técnica justificable.

12. Anexos y enlaces

Recurso	**Ubicación / enlace**
Repositorio GitHub	[ENLACE_REPOSITORIO]
Video de defensa	[ENLACE_VIDEO]
Anexo de evidencias	Anexo_Evidencias_TPI_MSII_Nievas_Figueroa.zip
Código fuente	Repositorio_TPI_MSII_Nievas_Figueroa.zip / carpeta del repositorio local
Checklist	docs/CODE_REVIEW_CHECKLIST.md
Uso de IA	docs/USO_DE_IA.md
Guion de video	docs/GUION_VIDEO.md

Materiales de referencia de la asignatura utilizados para mantener coherencia conceptual: Semana 1 - Calidad de Diseño y Construcción; Semana II - SOLID; Semana III - Clean Code y refactoring; materiales de Verificación y Validación, Repositorios e IA como Copiloto; consigna oficial del TPI y modelo orientativo de presentación.

Estado previo a la publicación. El contenido técnico, las pruebas, la documentación y el historial local se encuentran preparados. Antes de subir la entrega a la plataforma deben completarse tres datos externos que no pueden generarse localmente: comisión, URL real del repositorio GitHub y URL del video grupal.